

计算机基础与网络基础（AI 播客复习稿·融合版）

生成日期：2026-02-05。用途：睡前 30—60 分钟“听一遍”把 OS + 网络 + 网络 IO 的高频面试点串成一条线。

特点：每个主题都给“面试可复述的回答模板”+“常见追问”+“工程落地关键词”。

使用方式（强烈建议）

- 第一遍：顺着标题快速扫（10 分钟），只记每节 3 个关键词。
- 第二遍：当播客听（30—60 分钟）。听到卡顿处暂停，回看该节“回答模板”。
- 临面试前：只看文末《高频 25 问速记》。

你提供的资料摘录（用于对齐口径）

来自资料的关键表述（节选）：

- “用户态和内核态”
- “用户态和内核态的区别？”
- “内核态和用户态是操作系统中的两种运行模式。它们的主要区别在于权限和可执行的操作：”
- “内核态（Kernel Mode）：在内核态下，CPU 可以执行所有的指令和访问所有的硬件资源。这种”
- “内核态的底层操作主要包括：内存管理、进程管理、设备驱动程序控制、系统调用等。这些操作”
- “进程管理”
- “线程和进程的区别是什么？”

下面正文将这些表述与常见网络基础题型融合成一篇连贯的复习稿。

第一部分：操作系统高频（把“发生了什么”讲成流程）

1) 用户态 vs 内核态（系统调用入口题）

核心抓手：权限与隔离。用户态不能直接访问硬件/关键内存；需要通过系统调用陷入内核，由内核完成文件、网络、进程、内存等核心操作。

面试回答模板：

“用户态/内核态是 CPU 权限级别的划分。用户态权限受限，通过系统调用进入内核态访问硬件资源。这样做主要为了安全（防止越权）、稳定（单个应用崩溃不拖垮系统）、隔离与可维护性（边界清晰）。”

常见追问：系统调用会发生什么？

- 从用户态切到内核态：保存现场（寄存器/PC），切换到内核栈。
- 执行内核服务例程（比如 read/send/accept）。
- 返回用户态：恢复现场，继续执行。

2) 进程/线程/协程（并发模型三板斧）

一句话：进程是资源分配单位（地址空间/FD/权限等），线程是调度执行单位（共享进程资源但有独立栈与寄存器上下文），协程是用户态的更轻量执行单元（由 runtime 调度）。

面试回答模板：

“进程隔离强但切换成本更高；线程共享地址空间、切换更快；协程在用户态调度，创建/切换更轻量，但遇到阻塞系统调用仍需要内核线程支撑。”

常见追问：为什么线程切换比进程快？

同进程内线程共享地址空间，一般不需要切换页表（地址空间映射），只需保存/恢复寄存器、栈指针等线程上下文；进程切换还涉及页表切换与 TLB 失效等开销。

3) 进程状态与上下文切换（把“卡住了”定位到原因）

常见状态：创建→就绪→运行→阻塞→结束。I/O/锁等待导致运行→阻塞；事件完成导致阻塞→就绪；时间片用完导致运行→就绪。

上下文切换要说清“切什么”：寄存器/PC/栈；（进程级）页表/地址空间；内核态数据结构（PCB/TCB、内核栈）。

4) IPC（面试考“会选型”）

常见 IPC：管道（匿名/命名）、消息队列、共享内存、信号、信号量、Socket。

选型模板：

“同机大数据量：共享内存最快，但要配合信号量/锁做同步；跨机器：Socket；结构化异步消息：消息队列；父子进程简单通信：匿名管道。”

5) 锁、竞态与死锁（能讲清“为什么卡住”就赢）

竞态条件：多个执行流并发访问同一共享资源，且至少一个写，导致结果取决于执行时序。锁用于保证临界区互斥。

自旋锁 vs 互斥锁：

自旋锁失败后忙等（CAS）适合短临界区；互斥锁失败后阻塞让出 CPU 适合临界区较长。单核环境自旋锁需要配合抢占式调度避免“自旋到死”。

死锁四条件（必背）：互斥、持有并等待、不可剥夺、环路等待。

避免死锁最常用：资源有序分配/统一加锁顺序；必要时加超时与回退。

6) 内存管理（虚拟内存→页表→缺页一条链）

虚拟内存让每个进程看到独立地址空间，带来隔离、保护与更大可用空间。页表负责虚拟页到物理页映射，MMU 完成地址转换；缺页会触发异常进入内核，换入页面并更新页表后继续执行。

堆 vs 栈回答四维：分配方式（动态/自动）、增长方向（堆向上/栈向下，概念上）、速度（栈快）、碎片（堆更易碎片化）。

第二部分：网络基础高频（把“端到端链路”讲成分层）

1) 分层模型：TCP/IP 四层（面试够用）

- 链路层：以太网/WiFi，负责局域网内帧传输（MAC）。
- 网络层：IP，负责跨网段路由（IP 地址）。
- 传输层：TCP/UDP，负责端到端传输与复用（端口）。
- 应用层：HTTP、DNS、RPC 等。

回答模板：

“分层的价值是解耦：每层只关心自己的职责。排查问题时也能按层定位：先看链路（丢包/抖动）→再看 IP（路由）→再看 TCP（重传/拥塞）→最后看应用协议（超时/重试）。”

2) TCP 为什么可靠？（三件套）

- 序号/确认号：保证有序与确认；
- 重传：超时重传、快速重传；
- 滑动窗口：流量控制（接收端窗口）+ 拥塞控制（cwnd）。

追问：三次握手与四次挥手

三次握手建立连接（同步初始序号，确认双方收发能力）；四次挥手断开连接（半关闭，双方各自关闭发送方向）。

3) TIME_WAIT 为什么存在？怎么优化？

存在原因：保证最后 ACK 能重传；让旧连接的残留报文在网络中自然消失，避免污染新连接。

优化思路（面试说到这里就够）：短连接过多用连接复用/长连接；服务端可调内核参数但要理解风险；避免端口耗尽（扩大端口范围、减少主动关闭）。

4) DNS 与 HTTP/HTTPS（常见链路题）

- DNS：把域名解析为 IP，可缓存（本地/运营商/权威）。
- HTTP/1.1：默认长连接，但同域并发有限；HTTP/2：多路复用减少队头阻塞（应用层）。
- HTTPS：在 TCP 之上加 TLS 握手与加密，解决窃听/篡改/冒充。

面试落地说法：

“真实线上延迟常被 DNS、TLS、握手、慢启动影响。优化会优先做缓存与连接复用：DNS 缓存、HTTP keep-alive、TLS session resumption、以及请求合并/批处理。”

第三部分：网络 IO 与高并发（你资料里的重点）

你资料里把 IO 模型和

epoll/零拷贝作为最高频点（掌握/频次都很高），这一部分一面经常会问。

1) 五种 IO 模型（阻塞→异步）

根据资料整理：阻塞 IO、非阻塞 IO、IO 多路复用、信号驱动 IO、异步 IO。它们的差别主要在于：发起 IO 后是否阻塞、完成通知机制是什么。

面试回答模板：

“阻塞 IO 发起后线程一直等；非阻塞 IO 需要轮询；IO 多路复用用一个线程等待多个 fd 就绪（select/poll/epoll）；信号驱动由信号通知；异步 IO 提交请求后由内核完成并通知完成。”

2) 并发服务器的三种典型方式

- 多进程/多线程：简单但资源开销大；
- IO 多路复用：单线程/少线程处理大量连接（C10K）；
- 混合：多路复用 + 线程池/进程池（工程上最常见）。

3) select / poll / epoll：为什么 epoll 更适合高并发？

- select：fd 数量限制 + 每次调用需要拷贝/遍历；
- poll：无硬限制但仍需遍历；
- epoll：事件驱动，只返回就绪 fd；内核维护就绪队列，避免遍历全量。

4) epoll 的 LT/ET：怎么写才不丢事件？

LT（水平触发）：条件满足就可能多次通知，易写；ET（边缘触发）：状态变化只通知一次，需要非阻塞 + 循环读到 EAGAIN（把缓冲区读空），否则可能“漏读导致后续不再通知”。

面试回答模板：

“LT 稳妥简单；ET 性能更好但要求把数据一次性处理干净，通常要设置非阻塞并循环读写直到返回 EAGAIN。”

5) 零拷贝（sendfile）：为什么能更快？

资料给出的对比：传统路径有多次用户态/内核态切换和 CPU 拷贝；sendfile 把读文件与发网络合并为一次系统调用，减少上下文切换与拷贝次数，数据尽量在内核中完成传输。适合文件传输、日志、消息队列等。

工程追问：什么时候不用零拷贝？

当你需要在用户态修改/解析数据（比如加密、压缩、业务处理）时，零拷贝优势会下降；此外也要看内核/网卡能力与实际链路瓶颈。

第四部分：一面最实用的“排障叙事”

题型：‘延迟暴涨但 CPU 正常’、‘吞吐下降’、‘连接数上升’。你要会按层排查。

排查框架（从外到内）

- 应用层：超时/重试风暴？队列堆积？限流熔断是否触发？
- RPC/HTTP：连接池耗尽？keep-alive？P95 是否被少量慢请求拖高？
- TCP：重传率、RTT、拥塞窗口、TIME_WAIT/端口耗尽？
- 内核/IO：epoll 事件处理是否饥饿？线程阻塞在 IO/锁？fd 是否接近上限？
- 下游依赖：DB/向量库/Redis 的 P95、慢查询、热点 key/锁竞争。

面试回答模板：

“我会先看延迟与队列：如果 CPU 正常但 P95 上升，通常是 IO/锁/下游慢导致排队。然后按链路拆分：DNS/TLS/连接池、TCP 重传与 RTT、再到内核 fd/epoll 与线程阻塞，最后定位到具体依赖或热点。”

高频 25 问速记（睡前听完后，面试前 5 分钟看）

- 用户态/内核态区别？为什么要分？（安全/稳定/隔离 + 系统调用）
- 系统调用发生了什么？（陷入内核、切换内核栈、返回恢复）
- 进程 vs 线程 vs 协程？（资源/调度/用户态调度）
- 线程为什么比进程轻？（共享地址空间、少页表切换）
- 进程状态转换？运行→阻塞的原因有哪些？
- 上下文切换切什么？（寄存器/PC/栈 + 页表/PCB）
- IPC 有哪些？共享内存为什么快？为什么必须同步？
- 自旋锁 vs 互斥锁怎么选？（临界区长短/切换开销）
- 死锁四条件？怎么避免？（统一加锁顺序/超时）
- 虚拟内存好处？缺页异常流程？
- TLB 是什么？为什么快？
- 堆 vs 栈区别？（分配/增长/速度/碎片）
- TCP 可靠性靠什么？（序号确认/重传/窗口）
- 三次握手解决了什么？四次挥手为什么要四次？
- TIME_WAIT 为什么存在？怎么减少影响？
- DNS 解析过程与缓存层级？
- HTTP/1.1 vs HTTP/2 的关键差别？
- HTTPS (TLS) 解决什么问题？为何会变慢？怎么优化？

- 五种 IO 模型有哪些？IO 多路复用解决什么？
- select/poll/epoll 区别？为什么 epoll 更适合高并发？
- LT vs ET 区别？ET 为啥要非阻塞 + 读到 EAGAIN？
- 零拷贝是什么？sendfile 减少了什么？
- 服务 P95 延迟暴涨但 CPU 正常，你第一步看什么？
- 连接数暴涨，你会看哪些指标？（fd、TIME_WAIT、重传、连接池）
- 如何做 backpressure 防止写爆内存？（限流/队列/窗口/超时）

素材说明：本稿已阅读并融合你提供的两份资料（《操作系统面试题_QA整理.md》与《操作系统面试题 _ 小林coding.pdf》），并在其“网络 IO”部分基础上补齐了面试常见的 TCP/DNS/HTTP/HTTPS 基础链路。